
คู่มือโปรแกรมเมอร์

ในโปรแกรมนี้ได้แบ่งออกเป็น 3 ส่วนใหญ่ๆ ดังนี้

1. ส่วนการประมวลผล
2. ส่วนการแสดงผล
3. ส่วนจัดการฐานข้อมูล

ส่วนที่1 ส่วนการประมวลผลคือส่วนที่จัดทำเป็นเซิร์ฟเวอร์และมีการประมวลผลในฝั่งเซิร์ฟเวอร์

โปรแกรมนี้ได้สร้าง class ในลักษณะของ빈เพื่อใช้file JSP สามารถเรียกใช้งานกันได้ เป็นส่วนที่มีการโปรแกรมเพื่อรับภาพและทำการคำนวณหาค่า Histogram โดยมีฟังก์ชันการทำงานหลักๆอยู่ดังนี้

Coding Language : JAVA
Coding Tool : JBuilder, EditPlus

Class FileBmp

โดยคลาสนี้จะทำหน้าที่จัดการคำนวณแปลง file bmp ที่ได้รับ เพื่อค่าHistogram จะอยู่ในรูปแบบของbean ประกาศอยู่ใน package open ในการสร้าง package นี้เพื่อให้ง่ายในการจัดแบ่งกลุ่มของclass เพื่อให้มีการใช้งานที่ง่ายขึ้น

1. ส่วนการประกาศpackage และ ส่วน import class

```
package open;
import java.io.*;           เพื่อใช้งานกับ File
import java.lang.*;        เพื่อใช้งานกับ String
```

2. ส่วนภายในclass แบ่งออกเป็น 2 ส่วน

ส่วนที่1 ประกาศตัวแปร

int[] Histogram;	เก็บค่า ฮิสโตแกรมที่ได้จากการคำนวณ
byte[] FileData;	เก็บ file bmp ที่รับมา
byte[] Header;	เก็บ header ของ file bmp
byte[] ColorTable;	เก็บ color table หรือจานสี ของ filebmp
int[] ImageData;	เก็บ ค่าสี ของ file bmp ที่ทำการแปลงเป็นค่า RGB
String FileName;	เก็บ url file bmp
int bfSize;	เก็บขนาด filebmp
int bfOffBits;	เก็บตำแหน่งเริ่มต้นของข้อมูลภาพ
int biWidth;	เก็บขนาดความกว้างของภาพ
int biHeight;	เก็บขนาดความสูงของภาพ
int biBitCount;	เก็บจำนวนbit ต่อ1ค่าสี
int biSizeImage;	เก็บขนาดของภาพ

ส่วนที่ 2 ฟังก์ชันการทำงาน

constructor

```
- public FileBmp(){}
```

```
- public FileBmp(String filename){}
```

input : Type String
url file BMP

output : -
ใช้ในการset url ของ file BMPที่ต้องการใช้งานด้วย

Function & Method

- public int ReadHeader(String filename){}

input : Type String
url file BMP

output : Type int

return 0 : file not BMP file format

return 1 : file ok

return 2 : file data lost

other : error by exception form open and read file

ใช้ในการตรวจสอบว่าเป็น format filebmp หรือไม่ ถ้าใช้จะมีการเก็บข้อมูล header ของ file ไว้เพื่อใช้ในการคำนวณต่อไป

- public int ReadHeader(){}

input : -

output : Type int

return 0 : file not BMP file format

return 1 : file ok

return 2 : file data lost

other : error by exception form open and read file

ใช้ในการตรวจสอบว่าเป็น format filebmp หรือไม่ ถ้าใช้จะมีการเก็บข้อมูล header ของ file ไว้เพื่อใช้ในการคำนวณต่อไป

- public int GetFileData(){}

input : -

output : Type int

return 1 : file ok

other : hash code from error exception or read file error

ใช้ในการอ่านค่า filebmp ในส่วน ข้อมูลภาพ

- public int CreateImageData(){}

input : -

output : Type int

return 1 : file ok

other : hash code from error exception

ใช้ในการอ่านค่า filebmp ในส่วน ข้อมูลภาพ

- public int CalHistogram(int size){}

input : Type int

size ขนาดช่วงความถี่ของ Histogram

เช่น ต้องการ Histogram 64 ค่า size = 4 หรือ

Histogram 512 ค่า size = 8

Output : Type int

return 1 : file ok

other : hash code from error exception error

ใช้ในการคำนวณค่าHistogram รวมของภาพตามค่าจำนวนความถี่ที่ต้องการ

```
- public int GetHistogram(int i){}
    input      : Type int
                i คือค่าตำแหน่งของHistogram
    output     : Type int
                ค่า Histogram ณ ตำแหน่ง ที่ i
```

ใช้ในการแสดงผลค่า

```
- public int toInt(byte[] a){}
    input      : Type Array byte จำนวน 2 byte
    output     : Type int
```

ใช้ในการแปลงค่าจาก byte เป็นค่า int

```
- public short toShort(byte[] a){}
    input      : Type Array byte จำนวน 2 byte
    output     : Type short
```

ใช้ในการแปลงค่าจาก byte เป็นค่า short

```
- public int ByteF(byte a){}
    input      : Type byte
    output     : Type byte
```

ใช้ในการแปลงจากค่า byte ที่มีการรับค่ามาจากที่เป็นsign int ให้ เป็น ค่า byte ที่ถูกต้อง

Class SmartUpload

โดยClass นี้จะทำหน้าที่รับ fileจาก ผู้ใช้ผ่าน browser จะอยู่ในรูปแบบของbean ประกาศอยู่ใน package com.jspsmart.upload ซึ่งเป็นbean ที่ jspsmart เขียนขึ้น ซึ่ง class สามารถทำงานได้ทั้งในส่วน upload และส่วนในการ download ซึ่งเราใช้เฉพาะนั้น ส่วน upload จึงขอกล่าวเฉพาะในส่วนที่มีการใช้งานเท่านั้น

1. ส่วนการประกาศpackage และ ส่วน import class

```
package com.jspsmart.upload;
// Imports
import java.io.IOException;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Vector;
import javax.servlet.ServletConfig;
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import javax.servlet.jsp.JspWriter;
import javax.servlet.jsp.PageContext;
```

2. ส่วนภายในclass แบ่งออกเป็น 2 ส่วน

ส่วนที่1 ประกาศตัวแปร // Fields

```
protected byte[] m_binArray;
protected HttpServletRequest m_request;
```

```

protected HttpServletResponse m_response;
protected ServletContext m_application;
private int m_totalBytes;
private int m_currentIndex;
private int m_startData;
private int m_endData;
private String m_boundary;
private long m_totalMaxFileSize;
private long m_maxFileSize;
private Vector m_deniedFilesList;
private Vector m_allowedFilesList;
private boolean m_denyPhysicalPath;
private boolean m_forcePhysicalPath;
private String m_contentDisposition;
public static final int SAVE_AUTO = 0;
public static final int SAVE_VIRTUAL = 1;
public static final int SAVE_PHYSICAL = 2;
private Files m_files;
private Request m_formRequest;

```

ส่วนที่ 2 ฟังก์ชันการทำงาน

// Constructors

- public SmartUpload() { }

// Methods

ส่วนที่ใช้ติดต่อกับ http การจัดการserver และ servlet ซึ่งในส่วนนี้เราจะจัดการหรือไม่ก็ได้

```

- public final void init(ServletConfig servletConfig) throws
ServletException { }
- public void service(HttpServletRequest httpRequest,
HttpServletResponse httpResponse) throws IOException,
ServletException { }

```

```

- public final void initialize(ServletConfig servletConfig,
HttpServletRequest httpRequest, HttpServletResponse
httpServletResponse) throws ServletException { }
- public final void initialize(PageContext pageContext) throws
ServletException { }
- public final void initialize(ServletContext servletContext,
HttpSession httpSession, HttpServletRequest httpRequest,
HttpServletResponse httpResponse, JspWriter jspWriter)
throws ServletException { }

```

ส่วนนี้คือส่วนupload ที่เราสามารถเรียกดูค่าและ กำหนดค่าให้กับ bean class นี้ได้

```

- public void upload() throws SmartUploadException, IOException,
ServletException {}

```

เป็นคำสั่งที่ใช้ในการupload

```

- public int save(String url) throws SmartUploadException,
IOException, ServletException { }
input      : Type String

```

url ตำแหน่ง ที่ต้องการ save file
 output : Type int
 return 0 :save complete
 เป็นคำสั่งใช้บอกตำแหน่งที่ต้องการ save file

- public int save(String string, int int1) throws
 SmartUploadException, IOException, ServletException { return 0;}

- public int getSize() { return 0;}

input :-
 output : Type int
 return size ของ file นั้นๆ

ตัวอย่างการใช้งาน
 int size= mySmartUpload.GetFiles().getFile(0).getSize();
 เป็นการดูขนาดของ file

- public byte getBinaryData(int int0) { return 0;}

- public Files getFiles() { return null;}

input :-
 output : Type Files (เป็น type ที่สร้างขึ้นเฉพาะในการใช้งาน)
 return file ต่างๆที่ได้ upload มา
 เป็นการอ่าน file ที่ upload มา

- public Request getRequest() { return null;}

ส่วนที่เป็นการตั้งค่าให้ กับ file ซึ่งเป็นส่วนที่เราสามารถกำหนดค่าได้

- public void setDeniedFilesList(String sFile) throws SQLException,
 IOException, ServletException { }

input : Type String
 sFile คือ fileที่เราไม่อนุญาตในให้ upload
 output : -

ตัวอย่างการใช้งาน
 mySmartUpload.setDeniedFilesList("jpg,gif");
 เป็นการกำหนด type file ที่ไม่อนุญาตในการ upload

- public void setAllowedFilesList(String sFile) { }

input : Type String
 sFile คือ fileที่เราอนุญาตในให้ upload
 output : -

ตัวอย่างการใช้งาน
 mySmartUpload.setDeniedFilesList("bmp");

- public void setDenyPhysicalPath(boolean boolean0) { }

- public void setForcePhysicalPath(boolean boolean0) { }

- public void setContentDisposition(String string) { }

```

- public void setTotalMaxFileSize(long long0) { }
  input      : Type long
               long คือค่าของขนาดของทุกๆ file รวมกัน
  output     : -
ตัวอย่างการใช้งาน
               mySmartUpload.setTotalMaxFileSize(10000000);

- public void setMaxFileSize(long long0) { }
  input      : long
               long คือค่าขนาดของ file ไม่เกินค่าที่กำหนดนี้
  output     : -
ตัวอย่างการใช้งาน
               mySmartUpload.setMaxFileSize(10000000);

```

ขั้นตอนการใช้งาน bean

1. ทุก bean ควรอยู่ใน package
2. สร้าง folder ชื่อเดียวกันกับ package วางไว้ใน folder WEB-INF\classes
3. นำ class bean นั้นๆ ที่ทำการคอมไพล์เป็น .class แล้วไปวางใน folder package นั้นๆ

ขั้นตอนการทดลองการใช้งาน bean

เมื่อมีการแก้ไขคลาส bean จำเป็นต้อง restart server ทุกครั้ง

ส่วนที่ 2 ส่วนการแสดงผล คือส่วนที่รวม Interface ทั้งการรับข้อมูลและแสดงผล

ข้อมูล ในส่วนนี้เป็นการเขียนโปรแกรมเพื่อแสดงหน้าจอติดต่อกับผู้ใช้โดยมีหน้าตาหลักๆ ดังต่อไปนี้

1. หน้าตารับ input ของการค้นหาทั้ง 2 แบบ (โดยใช้ file BMP และใช้ keyword)
2. หน้าตาแสดงผลการค้นหา แบ่งตามลักษณะการค้นหาเป็น 2 ส่วน
 - หน้าตาการติดต่อกับฐานข้อมูลและแสดงผลการค้นหาโดยใช้ keyword
 - หน้าตาแสดงผลการรับข้อมูลภาพจากผู้ใช้ และแสดงเมนูการค้นหาโดยใช้ file BMP
 และ หน้าตาแสดงผลการค้นหาโดยใช้ file BMP เป็นภาพต้นแบบ และติดต่อกับฐานข้อมูล

Coding Language : JSP, Servlet, HTML

Coding Tool : JBuilder, EditPlus, Dreamweaver

Input

เป็นส่วนที่รับ input จากผู้ใช้

File HTML : SearchMenu.html

Code :

```

<html>
<head> <title>Search Manu</title> </head>
<body bgcolor="#FFEEFF">

```

```

<TABLE width="100%">

```

```

// ส่วนรับ keyword เป็น input

```

```

<TR>
  <TD width="20%"><CENTER>
    <FONT SIZE="" COLOR="#65338E">Keyword :< /FONT>
  </CENTER> </TD>
  <FORM METHOD=POST ACTION="keySearch.jsp"target="result">
    <TD width="50%"><INPUT TYPE="text" NAME="key"size="80"></TD>
    <TD ><CENTER><INPUT TYPE="submit"value="Search"></CENTER></TD>
  </FORM>
</TR>
//ส่วนรับ file BMP เป็น input
<TR>
  <TD width="20%"><CENTER>
    <FONT SIZE="" COLOR="#65338E">Image :</FONT>
  </CENTER> </TD>
  <FORM METHOD = POST ACTION= "imageSearch.jsp" ENCTYPE= "multipart/form-
    data" target="result">
    <TD width="50%" align="button"><INPUT TYPE="FILE" NAME="FILE1"
      SIZE="80">&nbsp;</TD>
    <TD ><CENTER><INPUT TYPE="submit" value="Upload"></CENTER></TD>
  </FORM>
</TR>

</TABLE>
</body>
</html>

```

Output #1

ส่วนในการแสดงผลการค้นหาด้วยkeywordและติดต่อฐานข้อมูล

File JSP : keySearch.jsp

Code :

ส่วนการตัดคำและนับจำนวนคำที่ตัดได้...

```

java.util.StringTokenizer sTok = new java.util.StringTokenizer(keyword);
int cnt = sTok.countTokens();
loop เท่ากับ cnt เพื่อหา sub key แต่ละชุดเพื่อใช้ในการค้นหา
subKey=sTok.nextToken();

```

ส่วนในการติดต่อฐานข้อมูล ...

ขั้นตอนที่1 โหลดคลาสไดรเวอร์ DriverName ขึ้นอยู่กับ DBMS ที่ใช้

```

//Class.forName("DriverName")
Class.forName ("org.gjt.mm.mysql.Driver");

```

ขั้นตอนที่ 2 เปิดการเชื่อมต่อกับฐานข้อมูล

```

//Connection connection = DriverManager.getConnection("url","userName","Password") ;
String url=" jdbc:mysql://161.246.6.46:3306/database";
Connection connection= DriverManager.getConnection(url,"user","password");

```

ขั้นตอนที่ 3 ติดต่อกับฐานข้อมูล

```

//Statement statement = conection.createStatement();
Statement statement = conection.createStatement();

```

รูปแบบในการติดต่อมี 2 รูปแบบ

แบบที่ 1 Query

```

//ResultSet resultset = statement.executeQuery("SQL Statement");
ResultSet resultset = statement.executeQuery("select * from his64 where
filename like '%" + subKey + "%'");

```

Or

```

ResultSet resultset = statement.executeQuery("select * from his512 where
filename like '%" + subKey + "%'");

```

แบบที่ 2 Update

```

//int return_result = statement.executeUpdate("SQL Statement");

```

ขั้นตอนที่ 4 ปิดการเชื่อมต่อกับฐานข้อมูล

```
//statement.close();
statement.close();
//connection.close();
connection.close();
```

loop เพื่อแสดงผลการค้นหา...

```
resultSet.next()
int id = resultSet.getInt("id");
String filename = resultSet.getString("filename");
String filepath = resultSet.getString("filepath");
ขั้นตอนแสดงภาพผลการค้นหา
out.println("<img src =/servlet/getImage?id="+id+">");
ขั้นตอนการแสดงผลชื่อภาพ
out.println("filename :"+filename);
```

Output #2

ส่วนในการแสดงผลภาพที่รับมาจากผู้ใช้และแสดงเมนูในการค้นหา

File JSP : imageSearch.jsp

Code :

ส่วน import ประกาศ Class ที่นำมาใช้งาน...

```
<%@ page import="com.jspsmart.upload.*"%>
<%@ page import="open.*"%>
<%@ page errorPage = "FileNotSupport.jsp"%>
```

ส่วนประกาศการใช้งาน Bean

ขั้นตอนในการประกาศ Bean นี้ใช้ในการupload file

```
//<jsp:useBean id="" scope="" class="" />
<jsp:useBean id="mySmartUpload" scope="page"
class="com.jspsmart.upload.SmartUpload" />
```

ขั้นตอนการประกาศ Bean ใช้ในการเก็บค่าชื่อ file ภาพที่มีการใช้งานใน Session เดียวกัน

```
<jsp:useBean id="myfile" scope="session" class="open.file"/>
```

ส่วนที่กำหนดค่าของการ upload ทำการ upload และจัดการภาพ ...

```
mySmartUpload.initialize(pageContext);
ขั้นตอนการกำหนดขนาด file รวมทั้งหมด
mySmartUpload.setTotalMaxFileSize(10000000);
ขั้นตอนการกำหนดประเภท file
mySmartUpload.setAllowedFilesList( "bmp");
ขั้นตอนการใช้คำสั่งการUpload
mySmartUpload.upload();
ขั้นตอนการใช้คำสั่งในการกำหนด ตำแหน่งที่ต้องการ save file
count = mySmartUpload.save("/fileupload");
ขั้นตอนการรับ file
```

```
com.jspsmart.upload.File myFile = mySmartUpload.getFiles().getFile(0);
```

ขั้นตอนการอ่านชื่อและขนาดของ file ที่รับเข้ามา

```
String filename=myFile.getFileName() ;
int size=myFile.getSize(); //int =32 bits
```

ขั้นตอนการ Set ค่า file ให้ Bean

```
myfile.setFileSize(size);*/
myfile.setFilename(filename);
```

ขั้นตอนการกำหนด ค่า Session เพื่อใช้สามารถมองเห็นค่านี้ในระดับ Session

```
session.setAttribute("filename",filename);
```

ส่วนการแสดงผลภาพที่รับจากผู้ใช้ และเมนูการ...

ขั้นตอนการแสดงผลรูปภาพและข้อมูลภาพ

```

<% out.println("File Name : " +filename + "<BR>");
```



```

        out.println("File Size  : " + mySmartUpload.getSize() + "<BR>");
        out.println("Content Type: " + myFile.getContentType() + "<BR>");  %>
ขั้นตอนการแสดงผลเมนูการค้นหา
        <FORM METHOD=POST ACTION="imageresult.jsp" target="result">
        HISTOGRAM:
        <input type="radio" name="his" value="his64" checked="his64">64
        <input type="radio" name="his" value="his512">512<br>
        <input type="hidden" name="filename" value="<%=myFile.getFileName()%>">
        <input type="submit" value = "Search by This Image File"><br><br>
        Histogram intersection value:
        <select name="exact">
        <option value="90">90%</option>
        <option value="80">80%</option>
        <option value="70">70%</option>
        <option value="60">60%</option>
        <option value="50">50%</option>
        <option value="40">40%</option>
        <option value="30">30%</option>
        <option value="20">20%</option>
        <option value="10">10%</option>
        </select>
        </FORM>

```

Output #3

ส่วนในการแสดงผลภาพที่ได้จากการค้นหาด้วยภาพและการติดต่อฐานข้อมูล

File JSP : imageResult.jsp

Code :

ส่วน import ประกาศ Class ที่นำมาใช้งาน...

```

<%@ page import ="java.io.*"%>
<%@ page import ="java.lang.String"%>
<%@ page import ="java.sql.*"%>
<%@ page import ="open.*"%>

```

ส่วนประกาศการใช้งาน Bean และ Set Page error

```

<jsp:useBean id="my" class="open.FileBmp" scope="page"/>
<%@ page errorPage ="FileNotFoundException.jsp"%>

```

```

<% String file=new String();
    file=(String)session.getAttribute("filename");

```

ส่วนนี้มีตรรกะตรวจสอบ File BMP และอ่านค่าเพื่อนำมาคำนวณหาค่า Histogram

ขั้นตอนในการตรวจสอบว่าเป็น FileBMPหรือไม่ และทำการอ่านHeader ของ file

```

// int =my.ReadHeader("url file BMP");
int error_report= my.ReadHeader ("C:/Program Files/Apache Tomcat 4.0/
webapps/ROOT/fileupload/"+file);

```

ขั้นตอนการ อ่าน File BMP ที่ตรวจสอบแล้วว่าถูกต้อง

```

if(my.GetFilesData()!=1){out.println("Get File Data Error");}

```

ขั้นตอนการแปลงข้อมูล File ดัชนี ให้ได้ค่า RGB ของ File BMP ที่สามารถนำไปคำนวณหาค่า Histogram ได้

```

error_report=my.CreateImageData();

```

ขั้นตอนการหารับค่า Histogram Intersect (=Exact)และ จำนวนความถี่ของค่า Histogram(=his)

จากผู้ใช้

```

String his=request.getParameter("his");
String exact=request.getParameter("exact");

```

ขั้นตอนการคำนวณหาค่า Histogram (จาก Parameter Exact และ his จากข้างต้น)

```

if (his.equals("his64")){
    error_report=my.CalHistogram(4);
}else{

```

```
error_report=my.CalHistogram(8); }%>
```

ส่วนนี้มีการแสดงภาพต้นแบบการ...

```
<TD></TD>
```

ส่วนนี้เป็นส่วนในการสร้างคำสั่งในการค้นหาข้อมูลและทำการติดต่อด้านข้อมูล...

ขั้นตอนที่ 1 โหลดคลาสไดรเวอร์

```
Class.forName("DriverName") ;
Class.forName("org.gjt.mm.mysql.Driver");
```

ขั้นตอนที่ 2 เปิดการเชื่อมต่อกับฐานข้อมูล

```
Connection connection = DriverManager.getConnection("url","Name","Password") ;
String dbpath="jdbc:mysql://161.246.6.46:3306/imagedb";
Connection connection=DriverManager.getConnection(dbpath,"user","");
```

ขั้นตอนที่ 3 ติดต่อกับฐานข้อมูล

```
Statement statement = conection.createStatement();
Statement statement = connection.createStatement();
```

ขั้นตอนการสร้างคำสั่งในการค้นหาจากฐานข้อมูล

```
String sql,sql2;
int concede=(100-Integer.parseInt(exact))*200000;
```

รูปแบบในการติดต่อกับ 2 รูปแบบ

แบบที่ 1 Query

```
ResultSet resultset = statement.executeQuery("SQL Statement");
if (his.equals("his64")){
    sql="select * from his64 where "+ concede + " >";
    for(i=0;i<64;i++){
        sql=sql+"abs(h"+i+"-"+(int) (((long)my.Histogram[i]*10000000)/size)+
            "+)";
        sql=sql+"0 order by 50000>abs(h0-2)";
    }
}else{
    sql="select * from his512 where "+concede+" >";
    for(i=0;i<512;i++){
        sql=sql+"abs(h"+i+"-"+(int) (((long)my.Histogram[i]*10000000)/size)+
            "+)";
        sql=sql+"0 order by 50000>abs(h0-2)";
    }
}
```

```
ResultSet resultset=statement.executeQuery(sql);
```

แบบที่ 2 Update

```
int return_resultset = statement.executeUpdate("SQL Statement");
```

ไม่มีการใช้งานในรูปแบบนี้

ขั้นตอนในการแสดงผลการค้นหา

```
rs.first();
int s=rs.getRow();
rs.last();
int e=rs.getRow();
out.println("Start:"+s+", End:"+e+",row count:"+e+"");
rs.beforeFirst();
int tis[]=new int[e];
int tid[]=new int[e];
int a,id,temp,h;
```

ขั้นตอนการวน Loop เพื่อคำนวณหาค่าHistogram intersection ระหว่างภาพต้นแบบที่ใช้ค้นกับภาพที่ได้จากการค้นหาจากฐานข้อมูลโดยเก็บ idของภาพกับค่า Histogram Intersectionของภาพต้นแบบกับภาพตาม id นั้นๆได้

```
while (rs.next()){
    a=rs.getRow();
    id=rs.getInt("id");
    temp=0;
    if (his.equals("his64")){
        for(int kk=0;kk<=63;kk++){
```

```

        h=(int)(((long)my.Histogram[kk]*10000000)/size);
        if((rs.getInt(kk+6))>(h)){
            temp=temp+h;
        }else{
            temp=temp+rs.getInt(kk+6);
        }
    }
} else{
    for(int kk=0;kk<=511;kk++){
        h=(int)(((long)my.Histogram[kk]*10000000)/size);
        if((rs.getInt(kk+6))>(h)){
            temp=temp+h;
        }else{
            temp=temp+rs.getInt(kk+6);
        }
    }
    tis[a-1]=temp ;
    d[a-1]=rs.getInt(1);
    t.println(id+" :"+a+",");
} rs.last();

```

ขั้นตอนการแสดงผลหากไม่พบภาพที่ต้องการค้นหา

```

if (rs.getRow()==0){
    out.print("<FONT COLOR=#0000CC > We are sorry, this file has not match in
    database </FONT>");
}

```

ขั้นตอนที่ 4 ปิดการเชื่อมต่อกับฐานข้อมูล

```

//resultset.colse();
//statement.close();
//connection.close();
resultset.close();
statement.close();
connection.close();

```

ขั้นตอนในการวน loop เพื่อเรียงลำดับภาพตามค่า Histogram Intersection ที่ทำการคำนวณไว้แล้ว เพื่อนำมาแสดงผลให้ผู้ใช้

ขั้นตอนการวนloop

```

int tempid;
int tempis;
for( i=0 ;i<e;i++){
    for(int j=i;j<e;j++){
        if (tis[i]<tis[j]){
            tempis=tis[i];
            tempid=tid[i];
            tis[i]=tis[j];
            tid[i]=tid[j];
            tis[j]=tempis;
            tid[j]=tempid;
        }
    }
}

```

ขั้นตอนในการแสดงผลให้ผู้ใช้

```

out.println("<TABLE >");
for(i=0;i<e;i++){
    out.println("<TR>");
    out.println("<TD >");
    out.println("Histogram intersection values :"+(tis[i]/100000)+" % <br>");
    out.println("id :"+tid[i]+"<br>");
}

```

```

        out.println();
        out.println("</TD>");
        out.println("<TD >");
        out.println("<img src =/servlet/getImage?id="+tid[i]+"><br>");
        out.println("</TD>");
        out.println("</TR>");
    }out.println("</TABLE>");

```

ปัญหาที่มักจะพบในส่วนที่2นี้คือ

1.ค่าตัวแปร int จะเป็น Sign int ในจาวา

ดังนั้นหากมีการใช้งานควรระวังในส่วนนี้

ส่วนที่ 3 ส่วนการจัดการฐานข้อมูลคือส่วนที่จัดการเก็บข้อมูลภาพและเก็บค่า Histogram

ส่วนนี้เป็นส่วนโปรแกรมในการเก็บข้อมูลHistogramของ ภาพลงในฐานข้อมูลแต่ในตัวอย่างนี้จะเป็นการเก็บโดยใช้ list ในการเก็บเพื่อให้เกิดความรวดเร็วในการเก็บข้อมูลภาพ

Coding Language : JAVA

Coding Tool : JBuilder, EditPlus

Class AdminInsert.java

1. ส่วนการประกาศ package และ ส่วน import class

```

package imagesearchapp;
import java.io.*;           เพื่อใช้งานกับ File
import java.lang.*;         เพื่อใช้งานกับ String
import java.sql.*;          เพื่อใช้งานกับ SQL Language
import org.gjt.*;           เพื่อใช้งานกับ MySQL

```

2. ส่วนภายในclass แบ่งออกเป็น 2 ส่วน

ส่วนที่ 1 Main Class

```

public static void main(String[] args){
    String driver="org.gjt.mm.mysql.Driver";
    String dbpath="jdbc:mysql://161.246.6.46:3306/database";
    String line;
    try{
        ขั้นตอนนี้เป็นส่วนที่รับและอ่านค่าใน List file เข้ามา (เฉพาะเป็น m3u File เท่านั้น)
        FileInputStream file=new FileInputStream("c:/image.m3u");

```

```

        BufferedInputStream bfile=new BufferedInputStream(file);
        DataInputStream in=new DataInputStream(bfile);

```

ขั้นตอนนี้ทำการติดต่อฐานข้อมูล

```

        Class.forName(driver);
        Connection connection;
        Connection= DriverManager.getConnection(dbpath,"root","style");
        Statement statement;
        ResultSet rs;
        String sql,sql2;
        statement=connection.createStatement();
        int id=0;

```

ขั้นตอนนี้วน loop รับค่า fileทีละบรรทัด แล้วนำมาตรวจสอบว่าเป็นค่า url ของ file BMP

หรือไม่

```

        while((line=in.readLine()) != null){

```

```

if(line.charAt(0)!='#'){
    id=id+1;
    line=bs2s(line);
    FileBmp my=new FileBmp();
    System.out.println(line);

    ขั้นตอนนีั้ตรวจสอบว่าเป็น FileBMP ถูกต้องแล้วจึงอ่านค่าHeader
    int errorreport=my.ReadHeader(line);
    ขั้นตอนนีั้รับค่าfile
    if(my.GetFileData()!=1){System.out.println("Get File Data Error");}
    ขั้นตอนนีั้คำนวณหาค่าRGB ของภาพ
    errorreport=my.CreateImageData();
    int size=my.biHeight*my.biWidth;

    ขั้นตอนนีั้คำนวณหาค่าHistogram 64และจัดเก็บลงในฐานข้อมูล
    errorreport=my.CalHistogram(4);
    ขั้นตอนนีั้สร้างคำสั่ง SQL
    sql=new String("insert into his64 (filepath,filename,type");
    sql2=new String(")values('"+line+"','"+filename(line)+"',24");
    int tmp;
    for (int i=0;i<64;i++){
        tmp=(int)(( ( (long)my.Histogram[i]) *10000000 )/size));
        sql=sql+"h"+i;
        sql2=sql2+"','"+tmp;
    } sql=sql+sql2+"";
    int ret=statement.executeUpdate(sql);

    ขั้นตอนนีั้คำนวณหาค่าHistogram512 และจัดเก็บลงในฐานข้อมูล
    errorreport=my.CalHistogram(8);
    ขั้นตอนนีั้สร้างคำสั่ง SQL
    sql="insert into his512 (filepath,filename,type";
    sql2=")values('"+line+"','"+filename(line)+"',24";
    for(int i=0;i<512;i++){
        tmp=(int)(( ( (long)my.Histogram[i]) *10000000 )/size));
        sql=sql+"h"+i;
        sql2=sql2+"','"+tmp;
    } sql=sql+sql2+"";
    ret =statement.executeUpdate(sql);
}
}

ขั้นตอนจบการเชื่อมต่อกับฐานข้อมูลและทำการปิด file ทั้งหมด
statement.close();
connection.close();
in.close();
bfile.close();
file.close();
System.out.println("count :"+id);

ขั้นตอนนีั้มีการตรวจสอบจับความผิดพลาดของโปรแกรม
}catch(FileNotFoundException e1){
    System.out.println("file not found");
}catch(IOException e2){
    System.out.println("IO error.");
}catch(Exception er){
    System.out.println(er.getMessage());
}
}

```

ส่วนที่ 2 Class Function

Function static public String filepath(String name){}

เป็นการหา filepath จาก url

Input : Type String
url

Output : Type String
Filepath

Code :
static public String filepath(String name){
int i=name.lastIndexOf("/");
return name.substring(0,i);
}

Function static public String filename(String name){}

เป็นการหา filename จาก url

Input : Type String
url

Output : Type String
Filename ชื่อของภาพ

Code :
static public String filename(String name){
int i=name.lastIndexOf("/");
return name.substring(++i);
}

Function static public String b2s(String name){}

เป็นการแปลงจาก \ เป็น /

Input : Type String
Output : Type String

Code :
static public String bs2s(String name){
char c='\\';
return name.replace(c,'/');
}